

Steganography Tool for Image/File Hiding

Complete Guide from Zero to Running Application

PART 1: CHECK IF PYTHON IS INSTALLED

Step 1: Open Command Prompt

- Press Windows Key + R
- Type: cmd
- Press Enter

Step 2: Check Python Version

```
python --version
```

Expected Output:

Python 3.11.x (or any 3.x version)

If you see error "python is not recognized":

- Go to <https://www.python.org/downloads/>
- Download latest Python (3.13.x or 3.13.y)
- **IMPORTANT:** During installation, CHECK the box "Add Python to PATH"
- Install Python
- Close CMD and open it again
- Try python --version again

PART 2: CREATE PROJECT FOLDER

Step 3: Navigate to Desktop

```
cd Desktop
```

Step 4: Create Project Folder

```
mkdir steganography_project
```

Step 5: Enter the Folder

```
cd steganography_project
```

Step 6: Verify You're in Correct Folder

```
cd
```

Expected Output: Should show path ending with \Desktop\steganography_project

PART 3: INSTALL REQUIRED LIBRARIES

Step 7: Install Pillow (Image Library)

```
pip install Pillow
```

Expected Output:

```
Collecting Pillow
```

```
  Downloading Pillow-10.x.x...
```

```
Installing collected packages: Pillow
```

```
Successfully installed Pillow-10.x.x
```

If you see "pip is not recognized":

```
python -m pip install Pillow
```

Step 8: Verify Pillow Installation

```
python -c "from PIL import Image; print('Pillow installed successfully!')"
```

Expected Output:

```
Pillow installed successfully!
```

PART 4: CREATE THE PYTHON FILE

Step 9: Create Python File

```
notepad steganography.py
```

A notepad window will open asking "Do you want to create a new file?"

- Click YES

Step 10: Copy-Paste Complete Code

Copy this ENTIRE code and paste into notepad:

```
import tkinter as tk

from tkinter import filedialog, messagebox, scrolledtext

from PIL import Image

import os


def message_to_binary(message):

    return "".join(format(ord(char), '08b') for char in message)

def embed_message(image_path, message, output_path):

    img = Image.open(image_path)

    if img.mode != 'RGB':

        img = img.convert('RGB')


    pixels = img.load()

    width, height = img.size

    message = message + "<<<END>>>"

    binary_message = message_to_binary(message)

    message_index = 0

    message_length = len(binary_message)


    for y in range(height):

        for x in range(width):

            if message_index < message_length:
```

```

    r, g, b = pixels[x, y]

    if message_index < message_length:

        r = (r & ~1) | int(binary_message[message_index])

        message_index += 1

    if message_index < message_length:

        g = (g & ~1) | int(binary_message[message_index])

        message_index += 1

    if message_index < message_length:

        b = (b & ~1) | int(binary_message[message_index])

        message_index += 1

    pixels[x, y] = (r, g, b)

else:

    break

if message_index >= message_length:

    break


img.save(output_path, 'PNG')

return True


def extract_message(image_path):

    img = Image.open(image_path)

    pixels = img.load()

    width, height = img.size

    binary_message = ""

```

```

for y in range(height):
    for x in range(width):
        r, g, b = pixels[x, y]

        binary_message += str(r & 1)

        binary_message += str(g & 1)

        binary_message += str(b & 1)

message = ""

for i in range(0, len(binary_message), 8):
    byte = binary_message[i:i+8]
    if len(byte) == 8:
        try:
            char = chr(int(byte, 2))
            message += char
            if message.endswith("<<<END>>>"):
                return message[:-9]
        except:
            continue
    return message

```

```

class SteganographyGUI:

    def __init__(self, root):

        self.root = root

        self.root.title("Steganography Tool")

        self.root.geometry("700x600")

```



```
self.decode_button.grid(row=0, column=1, padx=5)

self.content_frame = tk.Frame(self.root, bg="#f0f0f0")

self.content_frame.pack(pady=10, padx=30, fill=tk.BOTH, expand=True)

self.show_encode_tab()
```

```
def show_encode_tab(self):
```

```
    for widget in self.content_frame.winfo_children():
        widget.destroy()
```

```
    select_frame = tk.Frame(self.content_frame, bg="#f0f0f0")
    select_frame.pack(pady=15)
```

```
    select_btn = tk.Button(select_frame, text="📁 Select Image",
                           command=self.select_image_encode,
                           bg="#3498db", fg="white", width=20, height=2)
    select_btn.pack()
```

```
    self.image_label = tk.Label(self.content_frame, text="No image selected",
                                fg="#95a5a6", bg="#f0f0f0")
    self.image_label.pack()
```

```
    msg_label = tk.Label(self.content_frame, text="Enter Secret Message:",
                          font=("Arial", 11, "bold"), bg="#f0f0f0")
    msg_label.pack(pady=(20, 5))
```



```

        fg="#95a5a6", bg="#f0f0f0")

self.decode_image_label.pack()

decode_btn = tk.Button(self.content_frame, text="🔓 Extract Message",
                        command=self.decode_message,
                        bg="#e74c3c", fg="white", width=30, height=2,
                        font=("Arial", 11, "bold"))
decode_btn.pack(pady=20)

output_label = tk.Label(self.content_frame, text="Extracted Message:",
                        font=("Arial", 11, "bold"), bg="#f0f0f0")
output_label.pack(pady=(20, 5))

self.output_text = scrolledtext.ScrolledText(self.content_frame,
                                              height=12, width=70)
self.output_text.pack(pady=5)
self.output_text.config(state=tk.DISABLED)

def select_image_encode(self):
    file_path = filedialog.askopenfilename(
        title="Select Image",
        filetypes=[("PNG files", "*.png"), ("JPEG files", "*.jpg"),
                  ("All files", "*.*")] )
    if file_path:
        self.selected_image_path = file_path

```

```

        self.image_label.config(text=f"✓ {os.path.basename(file_path)}", fg="#27ae60")

def select_image_decode(self):
    file_path = filedialog.askopenfilename(
        title="Select Encoded Image",
        filetypes=[("PNG files", "*.png"), ("All files", "*.*)])
    if file_path:
        self.selected_image_path = file_path
        self.decode_image_label.config(text=f"✓ {os.path.basename(file_path)}",
                                         fg="#27ae60")

def encode_message(self):
    if not self.selected_image_path:
        messagebox.showerror("Error", "Please select an image first!")
        return

    message = self.message_text.get("1.0", tk.END).strip()
    if not message:
        messagebox.showerror("Error", "Please enter a message!")
        return

    output_path = filedialog.asksaveasfilename(
        title="Save Encoded Image",
        defaultextension=".png",
        filetypes=[("PNG files", "*.png")])

```

```

if output_path:

    try:

        embed_message(self.selected_image_path, message, output_path)

        self.status_label.config(

            text=f"✓ Success! Saved: {os.path.basename(output_path)}")

        messagebox.showinfo("Success", "Message hidden successfully!")

    except Exception as e:

        messagebox.showerror("Error", str(e))


def decode_message(self):

    if not self.selected_image_path:

        messagebox.showerror("Error", "Please select an image!")

        return

    try:

        message = extract_message(self.selected_image_path)

        self.output_text.config(state=tk.NORMAL)

        self.output_text.delete("1.0", tk.END)

        self.output_text.insert("1.0", message if message else "No message found")

        self.output_text.config(state=tk.DISABLED)

        messagebox.showinfo("Success", "Message extracted!")

    except Exception as e:

        messagebox.showerror("Error", str(e))


if __name__ == "__main__":

```

```
root = tk.Tk()

app = SteganographyGUI(root)

root.mainloop()
```

Step 11: Save the File

- In Notepad: Click **File** → **Save**
- **IMPORTANT:** Make sure it saves as steganography.py (not .txt)
- Close Notepad

Step 12: Verify File Was Created

dir

Expected Output: You should see steganography.py in the list

PART 5: PREPARE TEST IMAGE

Step 13: Download a Test Image

- Open your web browser
- Go to: <https://www.pexels.com/search/nature/>
- Download **ANY** image
- **IMPORTANT:** Save it as test.png in the steganography_project folder

OR Use this command to create a simple test image:

```
python -c "from PIL import Image; Image.new('RGB', (500, 500),
color='blue').save('test.png')"
```

Step 14: Verify Image Exists

dir

Expected Output: You should see both steganography.py and test.png

PART 6: RUN THE APPLICATION

Step 15: Run Your Steganography Tool

`python steganography.py`

Expected Result: A GUI window should open with your Steganography Tool!

If you see errors:

- Check if you installed Pillow: `pip install Pillow`
- Check if Python is in PATH: `python --version`

PART 7: TEST THE APPLICATION

Test 1: HIDE A MESSAGE

Step 16: In the GUI Window

1. Click "  **Select Image**" button
2. Browse to Desktop\steganography_project
3. Select test.png
4. Click **Open**

Step 17: Type Secret Message

- In the text box, type: This is my secret message!

Step 18: Hide the Message

1. Click "  **Hide Message**" button
2. Choose where to save (save as encoded_test.png)
3. Click **Save**
4. You should see success message!

Test 2: EXTRACT THE MESSAGE

Step 19: Switch to Decode Tab

- Click "🔒 Decode Message" tab

Step 20: Select Encoded Image

1. Click "📁 Select Encoded Image"
2. Select encoded_test.png (the file you just saved)
3. Click **Open**

Step 21: Extract Message

1. Click "🔒 Extract Message" button
2. You should see: This is my secret message!

PART 8: VIEW YOUR FILES

Step 22: List All Files in Folder

dir

You should now see:

steganography.py (your program)

test.png (original image)

encoded_test.png (image with hidden message)

Step 23: Open Images to Compare

start test.png

start encoded_test.png

Both images should look IDENTICAL to human eye!